WORD EMBEDDING

Word embeddings are dense numerical vector representations of words learned from text corpora such that words with similar meanings and contextual usage are located close to each other in a continuous vector space.

Purpose –

• Capture semantic similarity between words

• Reduce dimensionality compared to one-hot encoding

• Enable deep learning models to process text efficiently

• Improve generalization in NLP tasks

• Preserve syntactic and contextual relationships

Types –

1. **Frequency-Based (Count-Based) Embeddings**
   • Built using word occurrence statistics
   • Examples: Bag of Words, TF-IDF, Co-occurrence Matrix + SVD
2. **Prediction-Based (Neural) Static Embeddings**
   • Learned by predicting context using shallow neural networks
   • Examples: Google Word2Vec, Stanford University GloVe, Facebook AI Research FastText
3. **Contextual (Dynamic) Embeddings**
   • Word meaning changes depending on sentence context
   • Produced by deep transformer models
   • Examples: Google BERT, OpenAI GPT embeddings

ONE HOT ENCODING

It is a representation technique that converts each token into a sparse binary vector of length equal to vocabulary size. Even though, it does not capture any linguistic relationships, but act as starting point of many embedding methods.

Workflow of one hot encoding –

1.Corpus Collection — Gather raw text data that will be converted into numerical representations.

2.**Tokenization** — Split text into individual tokens (words) that act as categorical units.

3.**Vocabulary Construction** — Create a set of unique tokens to define the feature space.

4.**Index Mapping** — Assign each vocabulary word a unique integer ID for positional representation.

5.**Encoding Matrix Creation** — Build an identity matrix of size $V \times V$ where each row is a one-hot vector.

6.**Vector Lookup** — Represent each word by selecting its corresponding row from the encoding matrix.

7.**Sentence Encoding** — Stack word vectors to form a sparse feature matrix of shape $T \times V$.

8.**Model Input Preparation** — Use the encoded matrix as numerical input for ML/DL models.

9.**Embedding Layer Compatibility** — Multiply one-hot vectors with an embedding matrix to obtain dense word embeddings.

10.**Final Representation** — Produce sparse, high-dimensional vectors where one index is active and others are zero.

WORD2VEC

Word2Vec is a shallow neural network–based word embedding technique that converts words into dense vector representations such that semantically and syntactically similar words are positioned close to each other in a continuous vector space.

It is of 2 types –

- CBOW – It is a Word2Vec architecture that predicts a target word using the surrounding context words within a fixed window size.
- Skip- Gram – It is a Word2Vec architecture that predicts surrounding context words given a target word.

Workflow of CBOW

Step	Stage	Operation	Tensor Shape	Expalnation
1	Text Processing	Tokenization & vocabulary creation	—	Raw corpus is cleaned and converted into tokens with a vocabulary index.
2	Context Selection	Choose surrounding words of target	(C)	Select C context word indices using a sliding window.
3	One-Hot Encoding (Conceptual)	Convert context indices to one-hot vectors	$(C \times V)$	Each context word becomes a sparse vector of vocabulary size.
4	Embedding Lookup	$H = X W_{in}$	$((C \times V) \cdot (V \times d) = C \times d$	One-hot vectors select dense word embeddings from the input embedding matrix.
5	Context Pooling	$h = \text{mean}(H)$	$((C \times d) \rightarrow (1 \times d))$	All context embeddings are averaged to form a single semantic representation.
6	Output Projection	$u = h W_{out}$	$((1 \times d) \cdot (d \times V) = 1 \times V)$	Hidden vector is projected to vocabulary-sized logits.
7	Softmax	$\hat{y} = \text{softmax}(u)$	$(1 \times V)$	Converts logits into probability distribution over vocabulary.
8	True Label	Target word one-hot vector	$(1 \times V)$	Actual target word represented for loss computation.
9	Loss Computation	Cross-entropy loss	Scalar	Measures difference between predicted and true distributions.
10	Back-propagation	Gradient descent updates	$\left(W_{in}(V \times d), W_{out} (d \times V)\right.$	Model parameters are updated to improve embeddings.

Workflow of Skip-Gram

Step	Stage	Operation	Tensor Shape	Explanation
1	Corpus Processsing	Tokenization & vocabulary creation	—	Raw corpus is cleaned and converted into tokens with a vocabulary index.
2	Target Word Se-	Choose center word	(1)	Select a single central word from the context window.
3	Context Extraction	Pick C context word indices	(C)	Identify C surrounding words that fall within the context window.
4	One-Hot Encoding (Conceptual)	Convert target index to one-hot vector	(V)	Target word becomes a sparse vector of vocabulary size.
5	Embedding Lookup	$h = x W_{in}$	$((V) \cdot (V \times d) = d)$ d	One-hot vector selects dense word embedding from the input embedding matrix.
6	Output Projection	$u = h W_{out}$	$((d) \cdot (d \times V) = V)$	Hidden embedding is projected to vocabulary-sized logits.
7	Softmax	$\hat{y} = \text{softmax}(u)$	(V)	Hidden embedding is projected to vocabulary-sized logits.
8	Context Labels	2m one-hot context vectors	$(2m \times V)$	Converts logits into probability distribution over vocabulary.
9	Loss Computation	Sum cross-entropy loss	Scalar	Computes total difference between predicted and true contexts.
10	Back-propagation	Gradient descent updates	$W_{in}(V \times d)$, W_{out} $(d \times V)$	Model parameters are updated to improve embeddings.
10		Compact censor updates	$(W_n(V \times d),$ $W_{nn}(d \times V)$	Model parameters re updated to improve embeddings.

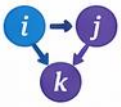
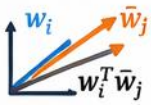
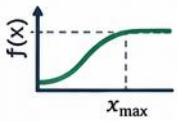
GLOVE

GloVe (Global Vectors) is a word embedding technique that learns dense vector representations of words by using global word co-occurrence statistics, so that words appearing in similar contexts have similar vector representations.

GloVe Mathematics — Complete Intuition & Formulas

How GloVe turns global co-occurrence counts into word embeddings

Goal: Learn word vectors whose dot products reconstruct logarithmic co-occurrence statistics.

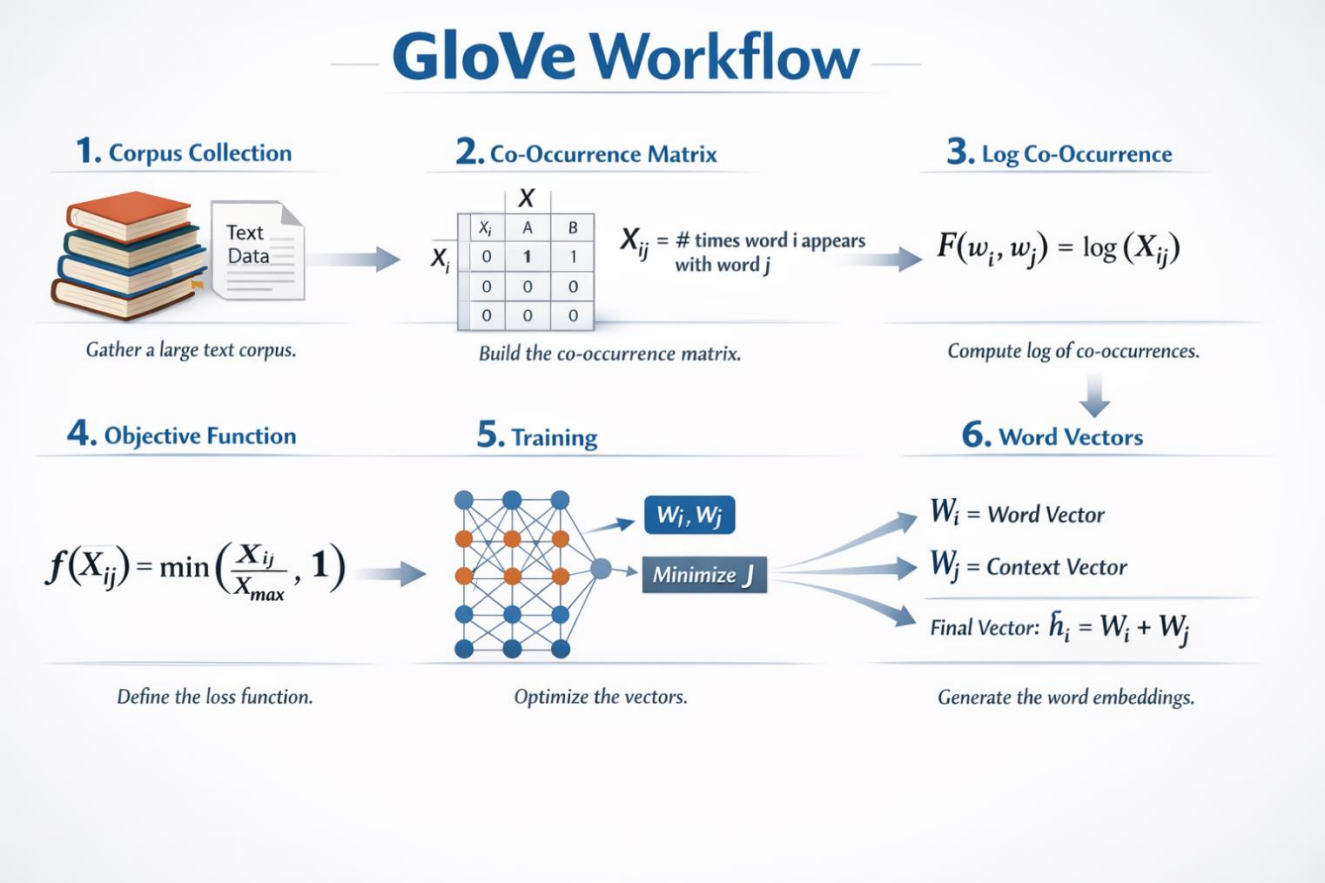
COMPONENT	MATHEMATICAL EXPRESSION	EXPLANATION
Co-occurrence Matrix	X_{ij} Count(word *j* in context of word *i*)	Number of times word *j* occurs in the context of word *i*. Global statistics from corpus
Co-occurrence Probability	$P_{ij} = \frac{X_{ij}}{\sum_k X_{ik}}$ Probability of *j* near *i*	Probability of observing word *j* near word *i*. Captures context distribution
 **Ratio of Probabilities**	$\frac{P_{ik}}{P_{jk}}$ How *i* and *j* relate to context *k*	Measures how differently words *i* and *j* associate with context word *k*. Encodes semantic relationships
 **Model Equation**	$w_i^T \bar{w}_j + b_i + b_j \approx \log(X_{ij})$ Word Vec · Context Vec + Biases ≈ Log Co-occurrence	Dot product + biases predict log co-occurrence count. Core GloVe insight
Loss Function	$J = \sum_{i,j} f(X_{ij}) \left(w_i^T \bar{w}_j + b_i + b_j - \log X_{ij}\right)^2$	Weighted least squares loss to minimize reconstruction error. Trains the embeddings
 **Weighting Function**	$f(x) = \begin{cases} (x/x_{max})^\alpha & x < x_{max} \\ 1 & \text{otherwise} \end{cases}$ ✓ Down-weights rare pairs ✓ Caps frequent pairs	Reduces impact of rare & extremely frequent pairs. Stabilizes training
Final Word Representation	$w_i^{final} = w_i + \bar{w}_i$ Element-wise sum of word & context vectors	Combined vector used as the final embedding. Used for downstream NLP tasks

Typical Hyperparameters

$x_{max} = 100$	$\alpha = 0.75$
Cutoff for weighting function	Smoothing factor

Key Insight: GloVe factorizes the global co-occurrence matrix so that word vector dot products approximate **log co-occurrence counts** — capturing semantics from **the entire corpus!**

Workflow of GLOVE



IMPORTANT POINTS

Technique	Best Use Case	Key Strength	Main Limitation
One-Hot Encoding	Very small NLP tasks and traditional ML models	Simple and requires no training	Produces sparse high-dimensional vectors with no semantic meaning
CBOW (Word2Vec)	Large datasets where fast training is required	Efficient and learns smooth semantic embeddings	Poor representation of rare words and ignores word order
Skip-Gram (Word2Vec)	Tasks needing good rare-word and semantic representations	Captures rich semantic relationships and handles rare words well	Slower training and higher computational cost
GloVe	When global corpus statistics and word analogies matter	Learns embeddings using global co-occurrence information with strong semantic structure	Requires large memory and is less suitable for very large or streaming corpora